

1

Context Engineering

This chapter introduces context engineering and explains why it has become a critical concept when building and using modern AI agents. While many AI systems are often described as "just prompts wrapped around an LLM," this chapter shows why that view breaks down as agents become more complex, long-running, and tool-driven. You will learn where context comes from, why it keeps growing, and how poor context handling leads to degraded performance, higher costs, hallucinations, and inconsistent behavior.

The goal of this chapter is to give you a clear mental model for context engineering and show how it is applied in practice. We start by explaining how context engineering evolved from prompt engineering, then use Claude Code as a concrete example of how modern agents write, select, compress, and isolate context. Towards the end, we will examine system prompts, why they still matter, and how to design them effectively.

NOTE

Your purchase includes a free PDF copy + code bundle

Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

The following topics will be covered in this chapter:

- Introduction to context engineering
- Claude Code and its context engineering philosophy
- System prompts and why they matter

NOTE

This discussion is based on publicly available information, including Anthropic's engineering blog posts and community analysis. It does not represent an official statement from Anthropic.

Introduction to context engineering

In this section, we will go into what context engineering is. Now, if you've been working with AI agents and probably coding agents such as Cursor and Claude Code, or maybe you have even developed AI agents for your companies or for your own use, you proba-

bly know that it basically all boils down to a prompt being sent to an LLM and a lot of engineering around it.

There is some truth in calling applications such as Cursor and Claude Code just wrappers around LLMs. However, building really good wrappers requires a lot of deep knowledge and a lot of engineering work. Another term often used for this surrounding system is an *agent harness*. The harness is the orchestration layer around the LLM. It is responsible for managing tool calls, controlling the agent loop, handling errors, enforcing guardrails, and, most importantly, deciding what context is sent to the model at each step.

In practice, most of the real engineering does not live inside the LLM call itself, but around it. The model invocation is often straightforward. What determines whether an agent works reliably is how the surrounding system manages state, tools, memory, and context. This is because calls to LLMs always come with context, and this context comes from various sources and ongoing processes.

For instance, context can come from multiple places:

- It can come from the developer of the application
- It can come from the user
- It can come from previous interactions of the user, tool calls, or other external data

Every day, new sources of context are added, and the amount of context keeps increasing. Sending the correct and relevant context to the LLM is not as simple as we thought it was in the early days.

From prompt engineering to context engineering

In the early days, we believed that prompt engineering was enough. We thought that writing some fancy prompts could fix the problem and give us what we want. The issue, however, is that prompts are static, while context is extremely dynamic.

If context is dynamic, then constructing the correct context requires a dynamic system as well. It is no longer just about writing a static prompt. This is why we are entering the realm of **context engineering**. It is the natural evolution of prompt engineering, but it is a much deeper concept.

Why context matters for agents

We all know the saying "*garbage in, garbage out*." This is one of the most common reasons why agentic systems do not perform the way they should. They are simply not provided with the right context.

LLMs cannot read our minds. We need to give them the right information. And by the way, it is not always just information or data. Sometimes we need to give them the correct tools so they can fetch other information, take actions, and perform tasks for us.

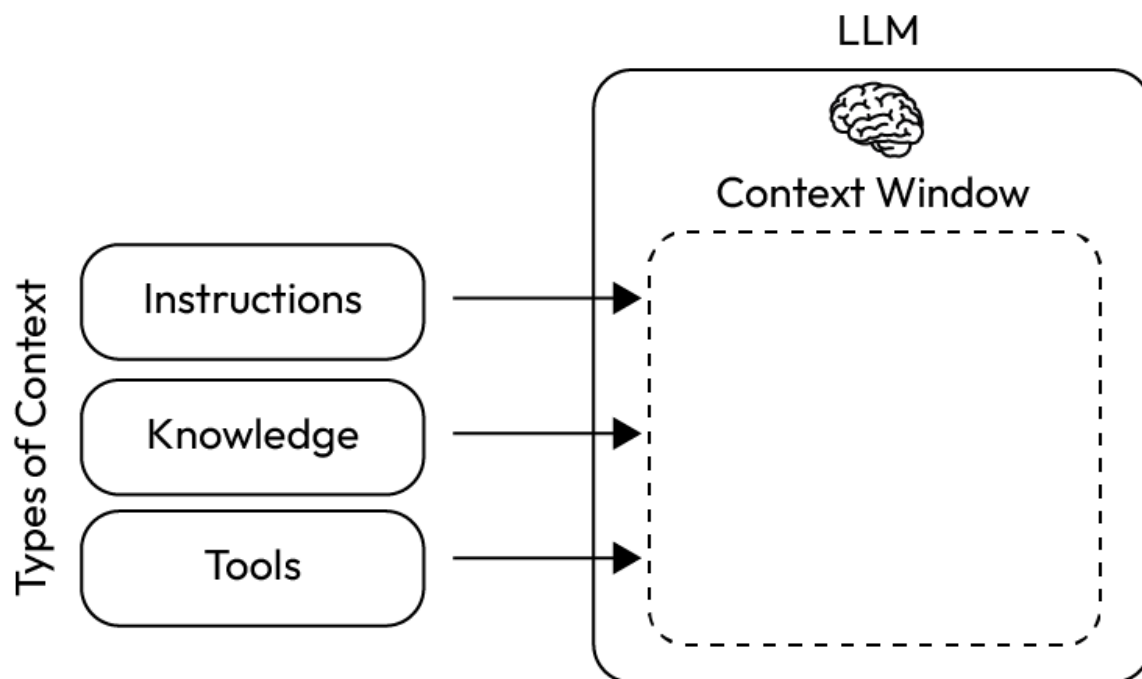


Figure 1.1 – LLM Context Window (Adapted from concepts discussed by LangChain, www.langchain.com)

LLMs are becoming better and better at reasoning. We have tool calling, and we can build AI agents that run tools, invoke them, get outputs, and loop until tasks are completed.

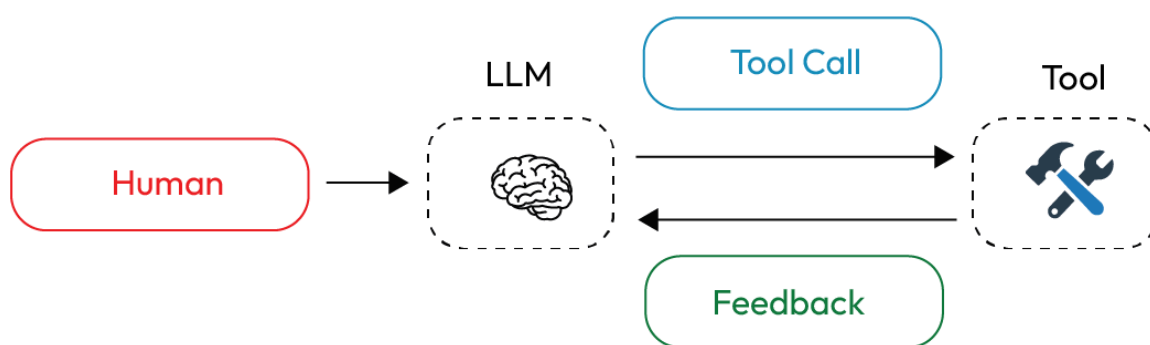


Figure 1.2 – LLM Tool Calling Loop (Adapted from concepts discussed by LangChain, www.langchain.com)

However, when it comes to long-running and complex tasks, we often accumulate feedback from tool calls.

This causes the context window to keep growing, filled with tool call results and intermediate outputs.

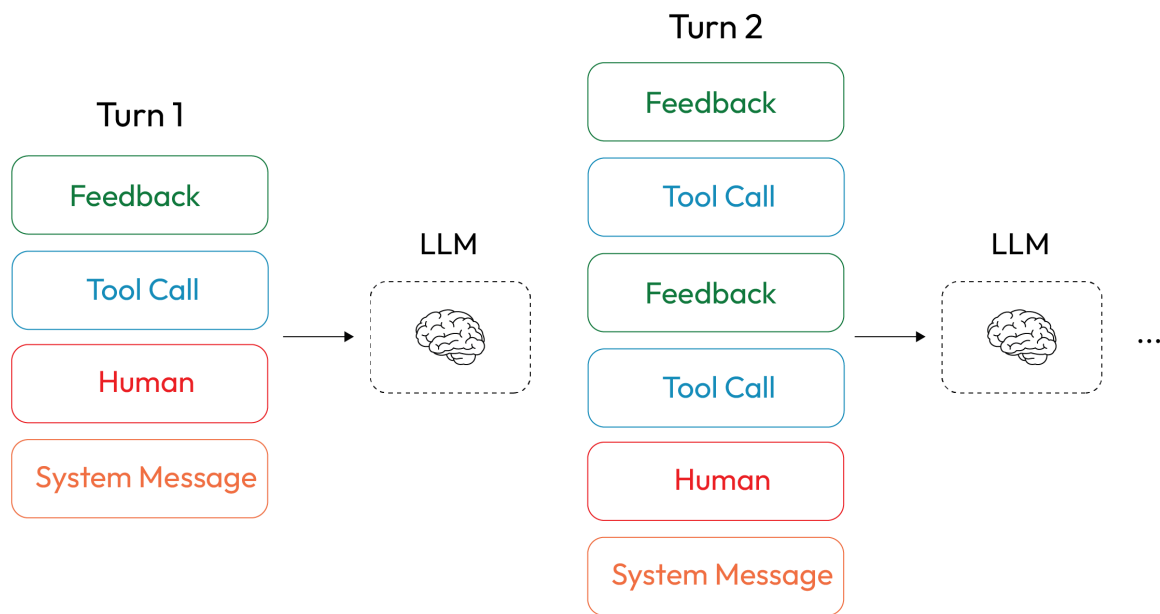


Figure 1.3 – Context Growth Across Multiple Turns (adapted from concepts discussed by LangChain, www.langchain.com)

This leads to several problems:

- The context window can exceed its size limit
- Cost and latency increase
- Agent performance starts to degrade

If nothing is done, this degradation becomes unavoidable. Recent discussions have explored why long contexts fail in practice and how degradation appears gradually as irrelevant or conflicting information accumulates. For a deeper discussion, see this great blog "[How Long Contexts](https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html)" by Dan Breunig (<https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html>). If context is allowed to grow without structure, selection, or control, then various problems will start to appear in agentic systems, such as the following:

- **Context poisoning:** When a hallucination from a tool call or LLM call enters the context and starts affecting future outputs
- **Context confusion:** When unnecessary context influences the response, even though it is not relevant to the task
- **Context clash:** When different parts of the context contradict each other

In the next section, we will discuss techniques for better context engineering. Some of these techniques are implemented by application developers, for example, in tools such as Claude Code. Other techniques are on the user side.

As users of Claude Code, we have a lot of influence over the context that is eventually sent to the LLM and the answers we receive. This means that even non-developers need

to understand context engineering if they want better responses from AI systems and AI agents.

Coding agents such as Claude Code are an excellent example of how context engineering techniques are applied on both the developer side and the user side. This is exactly what we will explore in the next section, as well as how to better engineer our context.

Claude Code and its context engineering strategies

In this section, we will talk about Claude Code, its context engineering philosophy, and how Claude Code addresses these challenges. Claude Code applies these context engineering strategies by implementing all four approaches, which are as follows:

- **Write Context:** Persistent memory systems
- **Select Context:** Intelligent context retrieval and injection
- **Compress Context:** Efficient context representation and summarization
- **Isolate Context:** Multi-agent and scoped context management

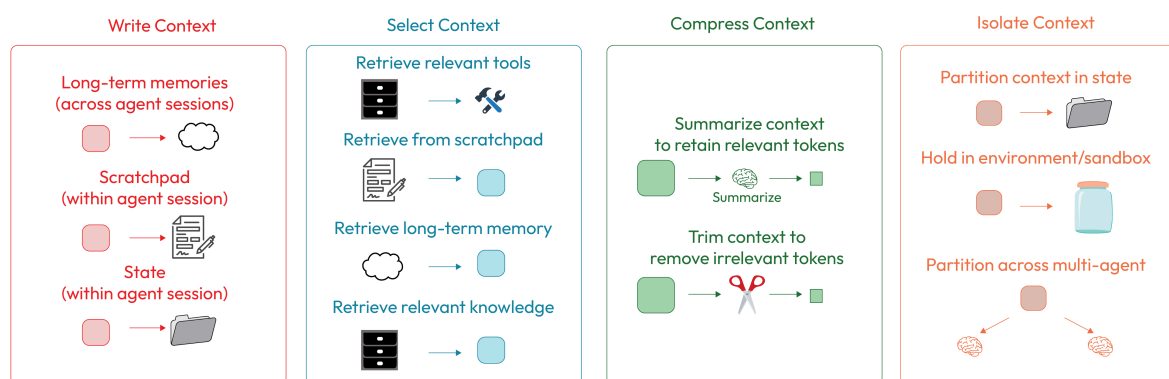


Figure 1.4 – Categories of context engineering (adapted from concepts discussed by LangChain, www.langchain.com)

Let's start with the first one.

Strategy 1: Writing context and persistent memory

The first strategy is **writing context and its persistent memory architecture**. Claude has a multilayered memory system, and Claude Code implements a three-tier memory hierarchy that persists context across sessions while coding with Claude Code.

- We first have the **project memory** (`./ CLAUDE.md`). This is a team-shared context for project architecture, standards, or anything related to a specific project. This kind of memory is version-controlled, and it is available to all team members.

Project Context

Architecture Overview

This is a microservices application using:

- Node.js with Express for API services
- React with TypeScript for frontend
- PostgreSQL with Prisma ORM
- Redis for caching

Coding Standards

- Use functional components with hooks
- Implement error boundaries for all route components
- Follow RESTful API conventions
- Write unit tests for all business logic

- Then we have the **user memory** (`~/. claude / CLAUDE.md`). This is stored in the home directory of the user, in the `claude` directory, in the `CLAUDE.md` file. This contains personal preferences and shortcuts across all projects for that user. It is not committed to GitHub and is user-specific. Every user will have different values here and it persists across all Claude Code sessions.

Personal Development Preferences

Code Style

- Always use explicit return types in TypeScript
- Prefer const assertions over type annotations
- Use descriptive variable names, avoid abbreviations

Workflow Shortcuts

- When writing tests, use Jest with React Testing Library
- Always run

`

`npm`

`run lint``

`before commits`

- Prefer composition over inheritance

- Lastly, we have **dynamic memory imports**. These allow us to import from other memory files using the `@` symbol and syntax. This is similar to loading regular context into Claude Code, but this time we can have dedicated memory files with specific information that we can reference inside our memory files.

```
# In any CLAUDE.md file
@path/to/memory/file.md
@./relative/path/context.md
@~/global/user/context.md
```

We can also customize behavior by writing scripts that dynamically update the context based on the Git branch. We can then connect this script to a context-switching hook and build more adaptive workflows.

One important detail to keep in mind is that we should avoid repeatedly appending the same context references to `CLAUDE.md`. If the script simply uses `> & gt ;` on every run, the file will grow indefinitely with duplicate entries. To prevent this, we add a small safeguard that checks whether a context reference already exists before appending it.

Below is an improved version of the script that avoids duplicate imports:

```
# Script to dynamically update context based on git branch
#!/bin/bash
# context-switcher.sh
# Dynamically load relevant context based on user query
# Safe against duplicate imports

CLAUDE_MD="CLAUDE.md"

# Create CLAUDE.md if it doesn't exist
touch "$CLAUDE_MD"

add_context() {
    local context_ref="$1"
    grep -qxF "$context_ref" "$CLAUDE_MD" || echo "$context_ref" >> "$CLAUDE_MD"
}

# --- Branch-based context ---
branch=$(git branch --show-current 2>/dev/null)

case $branch in
    "feature/auth-*")
        add_context "@./context/auth-system.md"
        ;;
    "feature/payment-*")
        add_context "@./context/payment-flow.md"
        ;;
    "hotfix/*")
        add_context "@./context/production-hotfix.md"
        ;;
    *)
        ;;
esac
```

```
# --- Query-based context ---
user_input="$1"

if [[ -n "$user_input" ]]; then
  if [[ $user_input == *"database"* || $user_input == *"migration"* ]]; then
    add_context "@./context/database-context.md"
  elif [[ $user_input == *"API"* || $user_input == *"endpoint"* ]]; then
    add_context "@./context/api-context.md"
  elif [[ $user_input == *"frontend"* || $user_input == *"component"* ]]; then
    add_context "@./context/frontend-context.md"
  fi
fi
```

The key change is the `add_context` helper function. It uses `grep - qxF` to check whether a context reference already exists in `CLAUDE.md`. If it does, nothing is appended. This makes the script idempotent and safe to run on every prompt submission.

We then connect this script to a hook

```
{
  "hooks": {
    "UserPromptSubmit": {
      "command": "./scripts/context-switcher.sh \"$PROMPT\""
      "description": "Dynamically load relevant context based on
        branch and user query"
    }
  }
}
```

With this setup, context switching becomes dynamic but remains stable over time.

Strategy 2: Intelligent context retrieval

The second strategy is **intelligent context retrieval**, usually implemented through dynamic context discovery.

Claude automatically looks through folders to find helpful context files. If we are in a subfolder, it also pulls context from parent folders, but it uses more specific information when it is available. It also prioritizes recently used and frequently accessed information.

This is context engineering at the application level, and Claude Code developers implement this logic. However, users can also add persistent context themselves using the `/memory` command. For example:

`/memory add Always use descriptive variable names`

Claude will prompt whether this memory should be stored at the project level or the user level, and it will update the appropriate `CLAUDE.md` file automatically. This allows users to influence future behavior without manually editing context files.

Based on the tool being used, different contexts are propagated to the LLM. For example, when Claude Code is about to edit a file, it automatically remembers things such as checking the existing code style first and looking for existing functions before creating new ones. This is the context that is relevant to the edit tool.

Another example is when Claude Code is about to run a terminal command, it remembers different things, such as checking if there is an existing `npm` script before running commands or making sure the file path exists before executing it.

Strategy 3: Compression of context

The third strategy is **compression of context**, which is about efficient context representation. Claude Code includes built-in compression commands.

- The `/clear` command resets the conversation history for the current context window while keeping the underlying project memory and user memory. It is useful when the running conversation has gone in an unhelpful direction or when you want to start a fresh interaction without losing what the system has already learned about the project.
- The `/compact` command summarizes the existing conversation into a shorter form. Instead of throwing everything away, it keeps the key decisions and important information while dropping minor details. This compressed history uses less of the context window, which reduces cost and latency, and gives the model more room for new inputs.

Strategy 4: Context isolation with subagents

The fourth strategy is context isolation. For this, Claude Code uses sub-agents. Each sub-agent runs in its own isolated context window and does not inherit the full conversation history of the main agent. This strategy is about having different specialist versions of Claude Code for different tasks, each with its own focused knowledge.

Instead of Claude Code trying to do everything at once with all the information, we create specialized sub-agents that act as experts in different areas. We can have the main Claude agent, which acts as a manager and delegates tasks. We can have a code review agent that focuses on code quality and security, a testing agent that focuses on writing

and running tests, and a research agent that focuses on finding information and best practices. Here is an example:

```
// Code review agent - focused context
Task(
  description: "Code review",
  prompt: "Review this PR focusing only on security and performance",
  subagent_type: "code-reviewer"
)

// Research agent - broad context
Task(
  description: "Research implementation",
  prompt: "Find best practices for OAuth2 implementation",
  subagent_type: "general-purpose"
)
```

This helps because, without isolation, Claude Code gets confused trying to do everything at once with all the possible information. With isolation, each specialist has a specific knowledge scope and uses it to do their job effectively.

NOTE

The preceding syntax is illustrative and simplified for demonstration purposes only. Claude Code does not currently use this exact format. In practice, defining subagents requires a YAML configuration with a frontmatter section, which we will cover later in the book.

System prompts and why they matter

In this section, we will discuss system prompts and their importance in context engineering. You have probably read a thousand times on X and LinkedIn that system prompts are important, and that you should work on them, iterate on them, and make them really good. Saying "have a good system prompt" is probably the most generic advice in AI engineering.

There are several public repositories that collect examples of system prompts used by well-known AI agents. These repositories typically aggregate prompts that have been publicly shared or discussed in public sources. Many of the examples focus on coding agents such as Claude Code, Cursor, and Devin, but they may also include prompts from other agent-style systems.

The purpose of referencing these repositories is not to verify the authenticity of every prompt, but to illustrate how large, structured, and carefully engineered system prompts tend to be in practice.

The goal of this section is not to analyze each prompt in detail or explain every technique used. We could have an entire book or course dedicated just to system prompts.

My point here is to show you that system prompts are important. System prompts keep evolving. As LLMs evolve, system prompts evolve with them. A lot of engineering effort goes into curating these prompts and making them better and better. This is an iterative process.

Best practices for crafting system prompts

Let us talk about some best practices when curating and crafting system prompts. It is like giving someone directions.

If we say something such as "go over there," they'll be confused. They won't know where to go. But if we give them a 50-page manual with every possible turn and street, we overwhelm them with information, and they still won't get where they need to go.

So, we want to be clear, we want to be specific, and we want to give just enough information to get them where they need to go. The hard part is finding that sweet spot.

The Goldilocks zone for system prompts

When we write system prompts, we are looking for what Anthropic refers to as the **Goldilocks zone**. Not too vague, not too detailed, but just right.

You can imagine this as a scale. On the far left, we have prompts that are too specific. On the far right, we have prompts that are too vague. What we want is right in the middle.



Figure 1.5 – Goldilocks Zone for System Prompts (Source: Anthropic, "Effective Context Engineering for AI Agents," <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>)

Let's break this down.

Problem with overly specific prompts

On the far left, we have very specific prompts. The core problem here is that we treat the LLM as a deterministic state machine instead of an intelligent agent. We hardcode logic.

For example, we might say: if the user's intent is incident resolution, ask three follow-up questions. *Why exactly three? What if two are enough? What if we need five?*

We also see exhaustive enumeration, where we try to list every possible escalation scenario. This is impossible to complete and forces the model down predetermined paths that might not match the actual input. It also becomes a maintenance nightmare, because every new edge case requires prompt changes.

At that point, if everything is predetermined, maybe we don't even need an autonomous agent. Maybe a deterministic workflow would be enough.

Problem with overly vague prompts

On the opposite end of the spectrum, we have very vague prompts. The core problem here is that they provide insufficient signal for consistent behavior.

In these prompts, we often see no actionable guidance. For example, consider the following prompt:

You are a bakery assistant. You should attempt to solve customers' issues in a manner consistent with the principles and essence of the company brand. Escalate to a human if needed.

What are those principles?

There is also a false assumption of shared context. The prompt assumes the model knows the company, the brand, and customer service norms, which it does not.

We also see undefined boundaries, such as "escalate to a human if needed." When is it needed? The model has no way to know.

There is no framework or structure for approaching problems systematically. This leads to inconsistent behavior, where different runs produce very different approaches to the same problem. Essentially, the prompt is saying "do the right thing" without defining what "right" actually means.

What a good middle-ground prompt looks like

Now let's look at the middle ground. Consider the following example:

You are a customer support agent for Claude's Bakery.

You specialize in assisting customers with their orders and basic questions about the bakery. Use the tools available to you to resolve issues efficiently and professionally.

You have access to order management systems, product catalogs, and store policies. Your goal is to resolve issues quickly when possible. Start by understanding the complete situation before proposing solutions, and ask follow-up questions if you do not understand.

Response Framework:

- 1. Identify the core issue – Look beyond surface complaints to understand what the customer actually needs*
- 2. Gather necessary context – Use available tools to verify order details, check inventory, or review policies before responding*
- 3. Provide clear resolution – Offer concrete next steps with realistic timelines*
- 4. Confirm satisfaction – Ensure the customer understands the resolution and knows how to follow up if needed*

Guidelines:

- When multiple solutions exist, choose the simplest one that fully addresses the issue*
- Check the status of an order if a user mentions it before suggesting next steps*
- Call the human assistance tool*
- For legal issues, health/allergy emergencies, or situations requiring financial adjustments beyond standard policies, call the human assistance tool*
- Acknowledge frustration or urgency in the user's tone and respond with appropriate empathy*

You can see the following:

- Here, we start by providing a clear identity and scope. This immediately establishes boundaries. It's customer support, not marketing or sales. It covers orders and basic questions, not complex business operations.
- We also empower rather than constrain. Instead of prescribing exactly which tool to use in every situation, we define a goal, such as efficient and professional resolution. The heuristic here is that we trust the agent to select appropriate tools when needed.
- We also provide a reasoning framework instead of a flowchart. A four-step response framework is being followed here: identify the core issue, gather necessary context, provide a clear resolution, and confirm customer satisfaction. This kind of framework should work across various scenarios, not rigid branching logic.
- Lastly, we have also established clear boundaries and principles. For example, if there are multiple solutions, choose the simplest one. This is a heuristic, not a rule, and it's similar to a greedy approach in computer science.

The overly specific prompt tries to do the LLM's thinking for it and breaks down when situations don't match the script. The vague prompt doesn't give the LLM enough to work with.

The middle prompt takes advantage of what modern LLMs are really good at: recognizing patterns and applying general principles to specific situations. It handles new situations well because it teaches principles instead of rules. It is efficient because it doesn't

waste words. Each guideline covers many scenarios. The principles are compressed, and there are no overlapping or contradictory instructions.

Summary

In this chapter, we introduced context engineering and explained why it is essential for building reliable and scalable AI agents. We examined how context differs from static prompts, where it comes from, and why unmanaged context leads to problems such as performance degradation, hallucinations, and inconsistent behavior. Using Claude Code as a practical example, we explored four core context engineering strategies: writing context through persistent memory, selecting relevant context dynamically, compressing context to keep it manageable, and isolating context using specialized sub-agents. We also discussed the role of system prompts, showing how effective prompts strike a balance between being too rigid and too vague. By now, you should have an idea of how modern AI agents manage context and how both developers and users can influence their behavior. In the next chapter, we will build on this foundation and begin working



step interactions.

In the next chapter, we will look into Claude Code and how it interacts with a codebase.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

